Test-Time Scaling

CSC3044S AI Systems

Lecturer: Dr Francois Meyer



Computer Science Department
University of Cape Town

LLM Agents Topic 2
6 October 2026

How to build CLAUDE CODE from scratch

1. Scale the LLM pretraining phrase
2. **Scale the LLM generation phase (test-time scaling)**
3. Enable the LLM to use files and tools
4. Wrap the LLM in an agentic framework

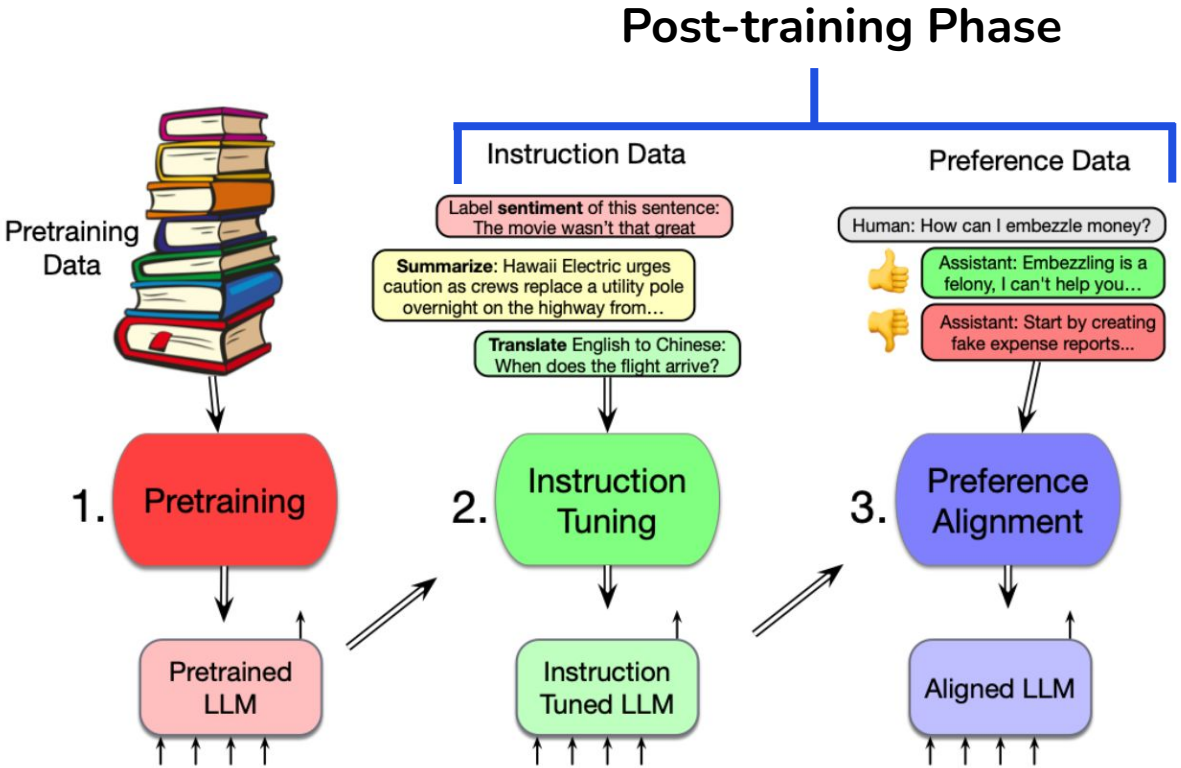
Pretraining

- Pretraining is the most computationally expensive part of developing LLMs.
- Scaling laws show that performance improves as a function of pretraining data size, model size, and compute.

Given a large pretrained LM, how can we “get the most out of it”?



Recap: LLM Training



NOTE: Post-training was covered in CSC3043F, so we will not discuss it here.

Remains an important part of LLM training – is usually used alongside methods we will cover today.

LLM Training

- **Instruction tuning:** teach to answer questions, format answers.

  (make it a chatbot)
- **Alignment:** refine behaviour and safety based on our preferences.

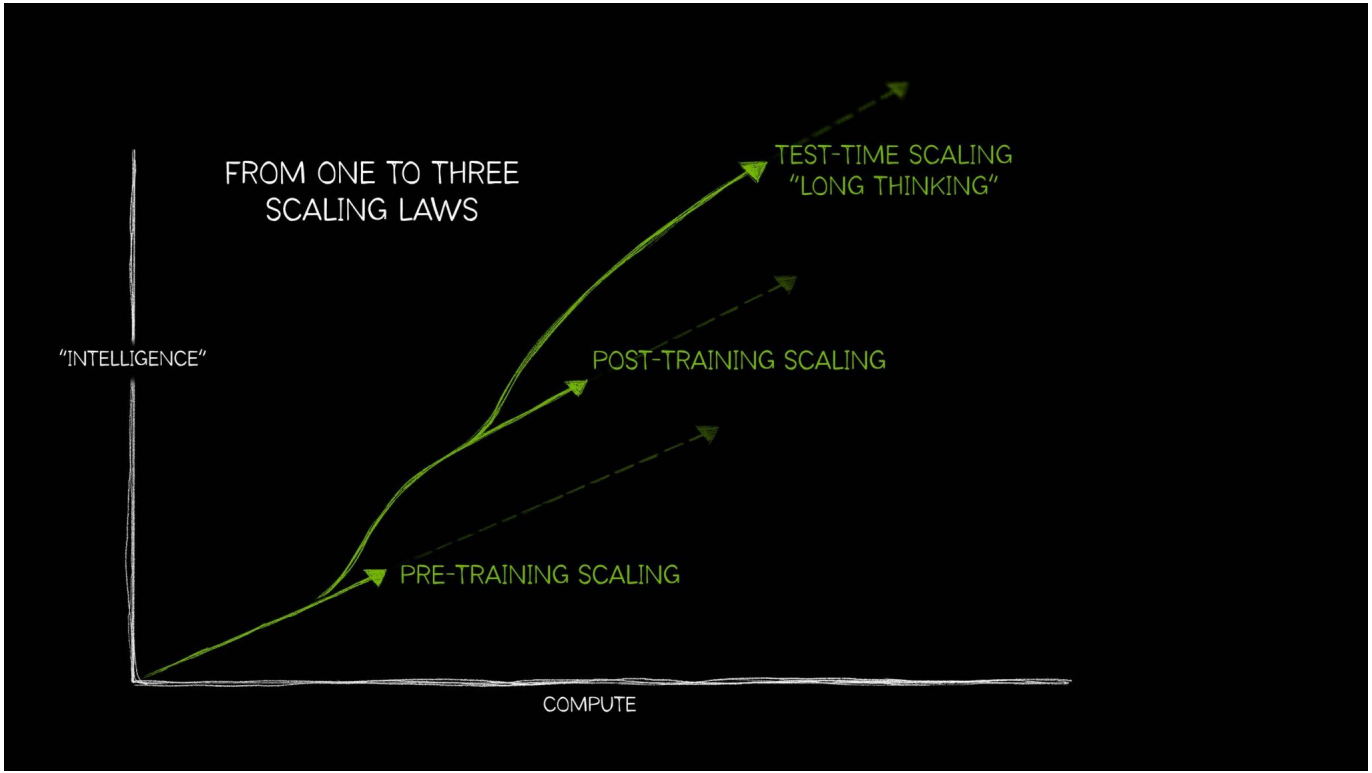
  (make it a *good* chatbot)

- **Test-time scaling:** spend more compute at inference to get better answers.

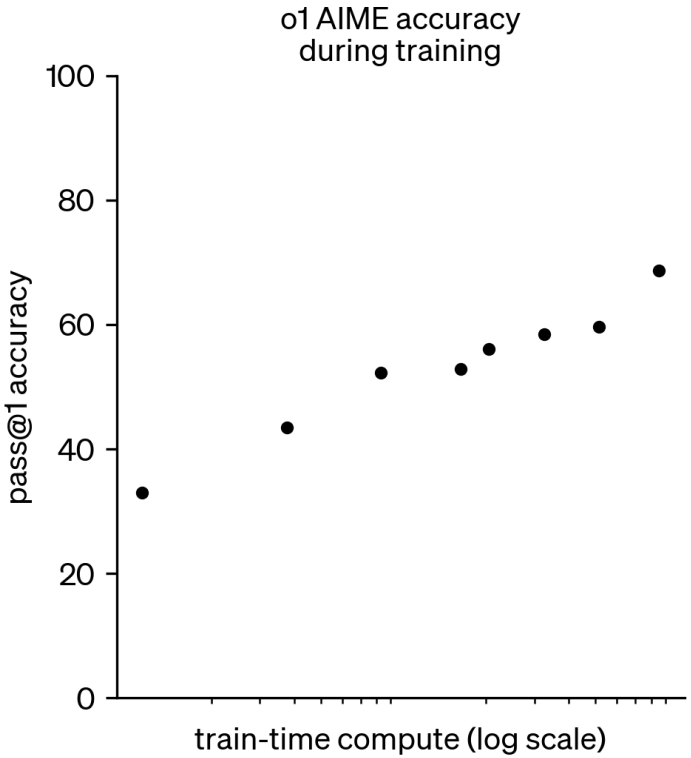
  Enables reasoning: work through problems rather than answer immediately.

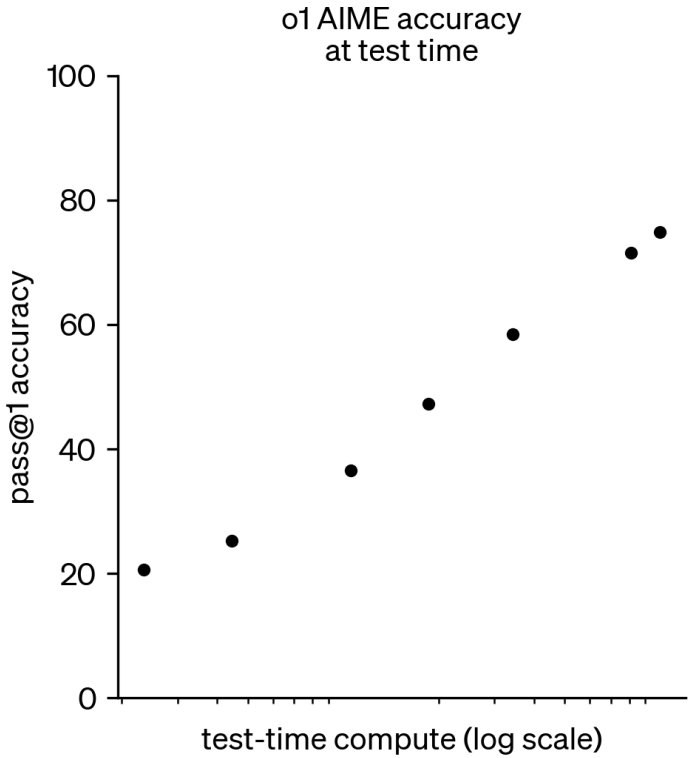
  Test-time scaling unlocks new capabilities – it allows the model to solve more complex tasks that require reasoning e.g. maths, coding.

LLM Training



OpenAI o1


o1 AIME accuracy
during training
100
80
60
40
20
0
pass@1 accuracy
train-time compute (log scale)


o1 AIME accuracy
at test time
100
80
60
40
20
0
pass@1 accuracy
test-time compute (log scale)

LLM Evaluation

Evaluation

How do we know how capable LLMs are?

Evaluation datasets containing questions and correct answers!

- **Factual knowledge:** answer questions without access to external data.

```
What is the capital city of Ghana? Accra
```

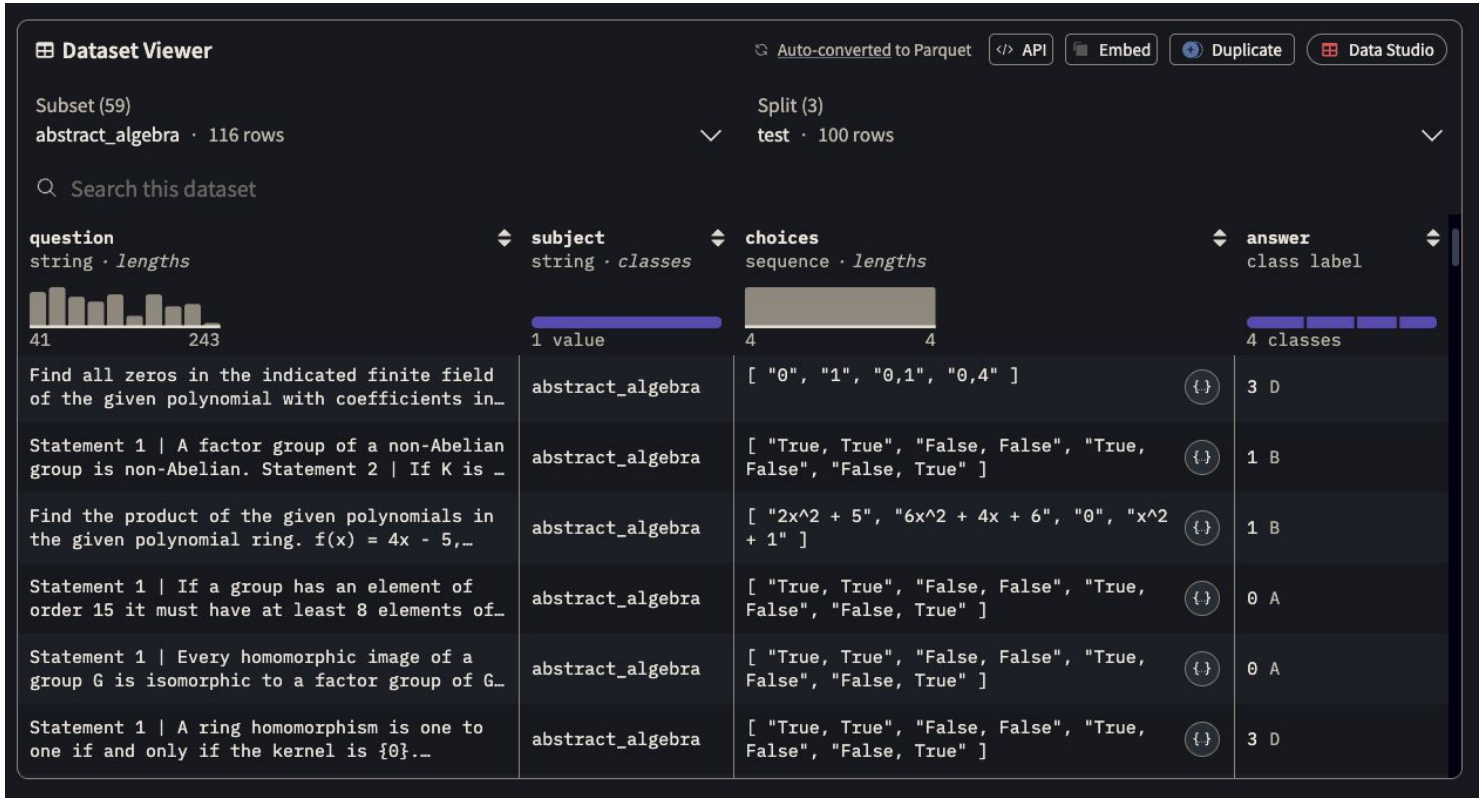
- **Multiple choice questions** test commonsense and reading comprehension.

```
Hat-trick-hero Salah helps Liverpool to 5-0 win over
United. How many goals did Salah score in this game?

A) 0      B) 1      C) 2      D) 3

D) 3
```

Evaluation


Dataset Viewer
Auto-converted to Parquet
API
Embed
Duplicate
Data Studio
Subset (59)
abstract_algebra · 116 rows
Split (3)
test · 100 rows
Search this dataset
question
string · lengths
41
243
subject
string · classes
1 value
choices
sequence · lengths
4
4
answer
class label
4 classes
Find all zeros in the indicated finite field of the given polynomial with coefficients in…
abstract_algebra
["0", "1", "0,1", "0,4"]
3 D
Statement 1 | A factor group of a non-Abelian group is non-Abelian. Statement 2 | If K is …
abstract_algebra
["True, True", "False, False", "True, False", "False, True"]
1 B
Find the product of the given polynomials in the given polynomial ring. f(x) = 4x - 5,…
abstract_algebra
["2x^2 + 5", "6x^2 + 4x + 6", "0", "x^2 + 1"]
1 B
Statement 1 | If a group has an element of order 15 it must have at least 8 elements of…
abstract_algebra
["True, True", "False, False", "True, False", "False, True"]
0 A
Statement 1 | Every homomorphic image of a group G is isomorphic to a factor group of G…
abstract_algebra
["True, True", "False, False", "True, False", "False, True"]
0 A
Statement 1 | A ring homomorphism is one to one if and only if the kernel is {0}.…
abstract_algebra
["True, True", "False, False", "True, False", "False, True"]
3 D

Evaluation

How do we evaluate more advanced capabilities in LLMs?

Datasets with question-answer pairs that require reasoning e.g. maths exams:

Problem
The battery charge in Mary's cordless vacuum cleaner lasts ten minutes. It takes her four minutes to vacuum each room in her house. Mary has three bedrooms, a kitchen, and a living room. How many times does Mary need to charge her vacuum cleaner to vacuum her whole house?

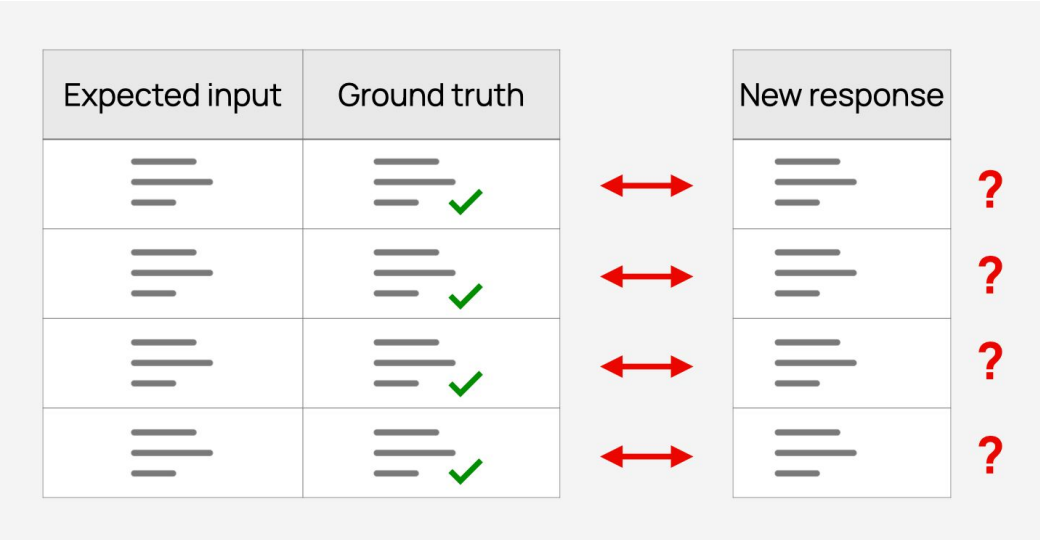
Solution
Mary has 3 + 1 + 1 = 5 rooms in her house.
At 4 minutes a room, it will take her 4 * 5 = 20 minutes to vacuum her whole house.
At 10 minutes a charge, she will need to charge her vacuum cleaner 20 / 10 = 2 times to vacuum her whole house.

Final Answer
2

Evaluation

Reasoning tasks:

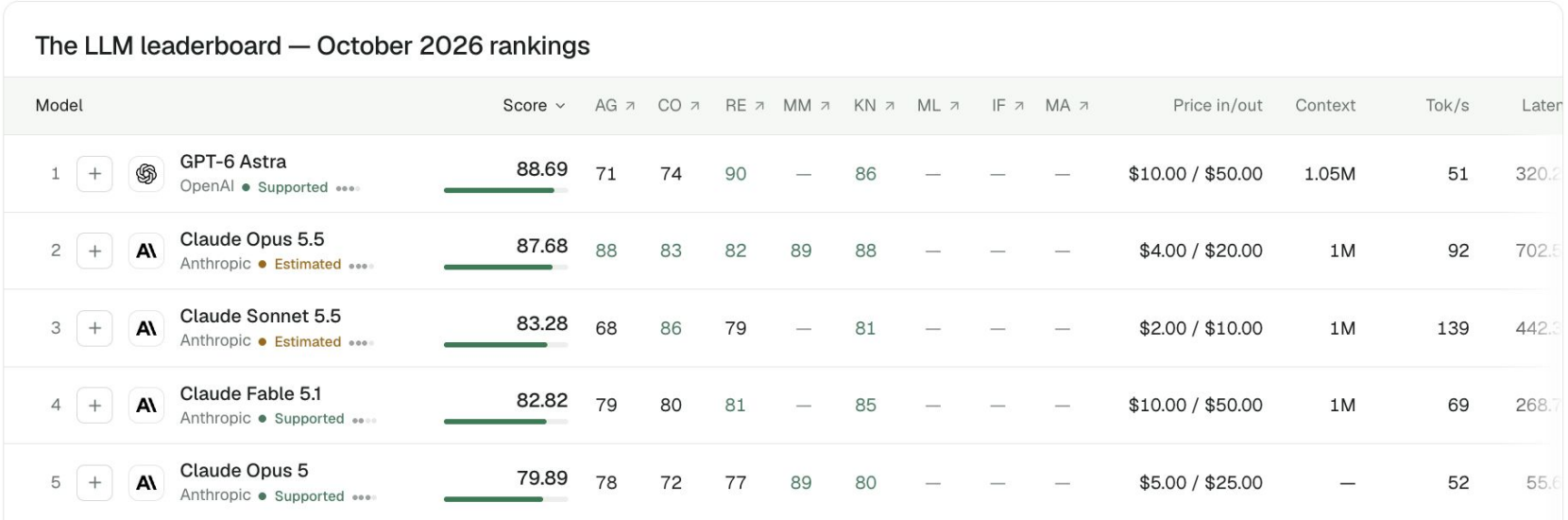
- Maths exam problems
- Commonsense reasoning
- Symbolic reasoning
- Hard science questions
- **Coding tasks** – can be evaluated by whether or not unit tests pass!



Evaluation

LLM evaluation is important, since it's how we track of AI progress.

This is what **benchmarks** are for – collections of datasets that quantify model performance across different tasks / capabilities.



The LLM leaderboard — October 2026 rankings

	Model	Score	AG	CO	RE	MM	KN	ML	IF	MA	Price in/out	Context	Tok/s	Later
1	GPT-6 Astra OpenAI • Supported	88.69	71	74	90	—	86	—	—	—	$10.00 / $50.00	1.05M	51	320.2
2	Claude Opus 5.5 Anthropic • Estimated	87.68	88	83	82	89	88	—	—	—	$4.00 / $20.00	1M	92	702.5
3	Claude Sonnet 5.5 Anthropic • Estimated	83.28	68	86	79	—	81	—	—	—	$2.00 / $10.00	1M	139	442.3
4	Claude Fable 5.1 Anthropic • Supported	82.82	79	80	81	—	85	—	—	—	$10.00 / $50.00	1M	69	268.7
5	Claude Opus 5 Anthropic • Supported	79.89	78	72	77	89	80	—	—	—	$5.00 / $25.00	—	52	55.6

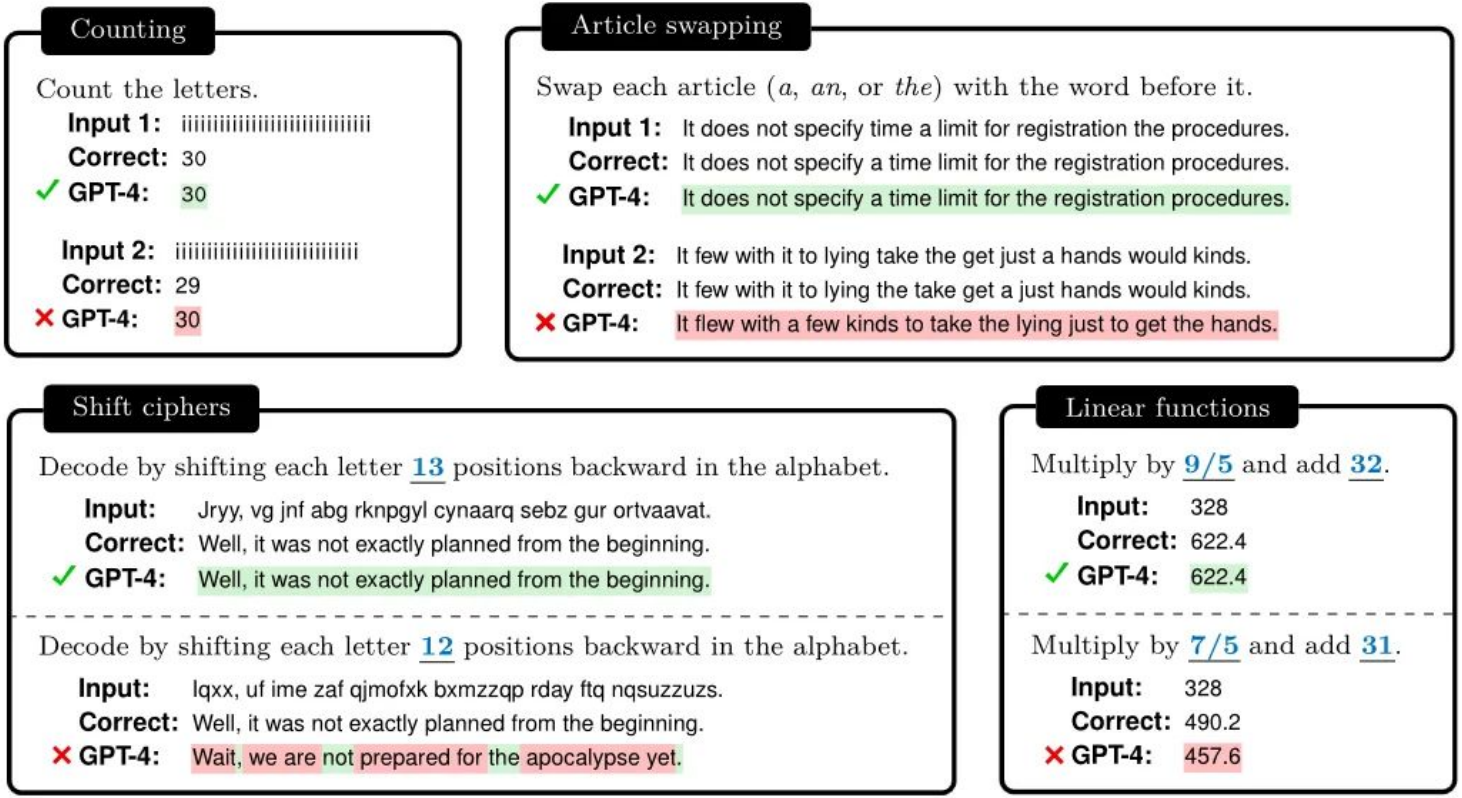
Evaluation

But LLM evaluation is becoming increasingly difficult.

- **Benchmarks saturate:** the field is advancing so fast that models get 100% accuracy on a new dataset before long.
- **Training data contamination:** benchmarks are public so they end up in the pretraining corpus (models might be memorising, not learning capabilities).
- **Complex capabilities are hard to evaluate!** Open-ended tasks have no straightforward right/wrong answer.

  How do we decide “is this a good research summary?” or “did the agent handle the customer well?”.

Evaluation


Counting
Count the letters.
Input 1: iiiiiiiiiiiiiiiiiiiiiiiiiiiii
Correct: 30
✓ GPT-4: 30
Input 2: iiiiiiiiiiiiiiiiiiiiiiiiiiiii
Correct: 29
✗ GPT-4: 30
Article swapping
Swap each article (a, an, or the) with the word before it.
Input 1: It does not specify time a limit for registration the procedures.
Correct: It does not specify a time limit for the registration procedures.
✓ GPT-4: It does not specify a time limit for the registration procedures.
Input 2: It few with it to lying take the get just a hands would kinds.
Correct: It few with it to lying the take get a just hands would kinds.
✗ GPT-4: It flew with a few kinds to take the lying just to get the hands.
Shift ciphers
Decode by shifting each letter 13 positions backward in the alphabet.
Input: Jryy, vg jnf abg rknpgyl cynaarq sebz gur ortvaavat.
Correct: Well, it was not exactly planned from the beginning.
✓ GPT-4: Well, it was not exactly planned from the beginning.
Decode by shifting each letter 12 positions backward in the alphabet.
Input: lqxx, uf ime zaf qjmofxk bxmzzqp rday ftq nqsuzzuzs.
Correct: Well, it was not exactly planned from the beginning.
✗ GPT-4: Wait, we are not prepared for the apocalypse yet.
Linear functions
Multiply by 9/5 and add 32.
Input: 328
Correct: 622.4
✓ GPT-4: 622.4
Multiply by 7/5 and add 31.
Input: 328
Correct: 490.2
✗ GPT-4: 457.6

What is test-time / LLM inference?

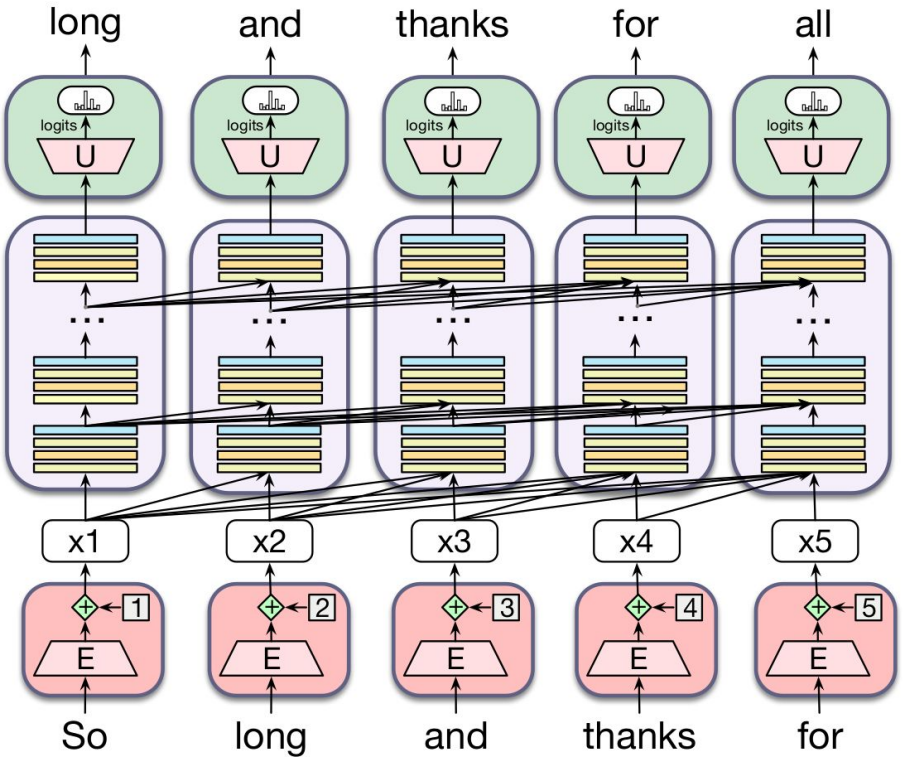
LLM Pretraining

Dataset: large dataset of **raw text** (e.g. web-scraped corpus)

Batch: continuous **text sequences** sampled from dataset

Task: **predict the next word**, given all preceding words in sequence (we use **teacher forcing**).

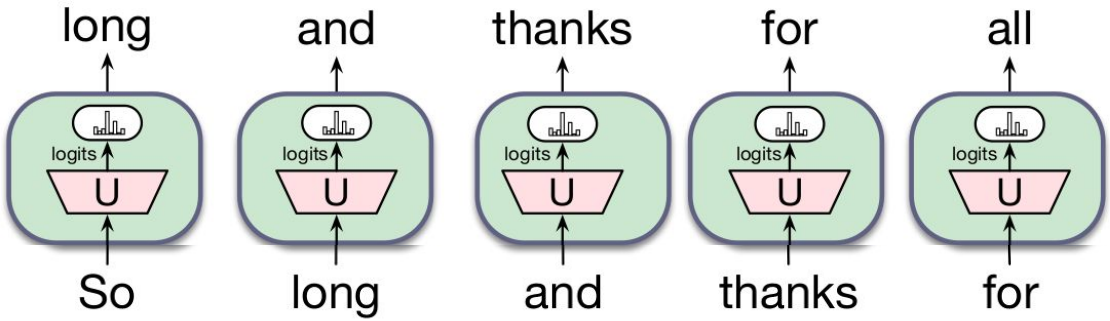
Loss: **cross-entropy** loss summed over all tokens a sequence.



LLM Inference

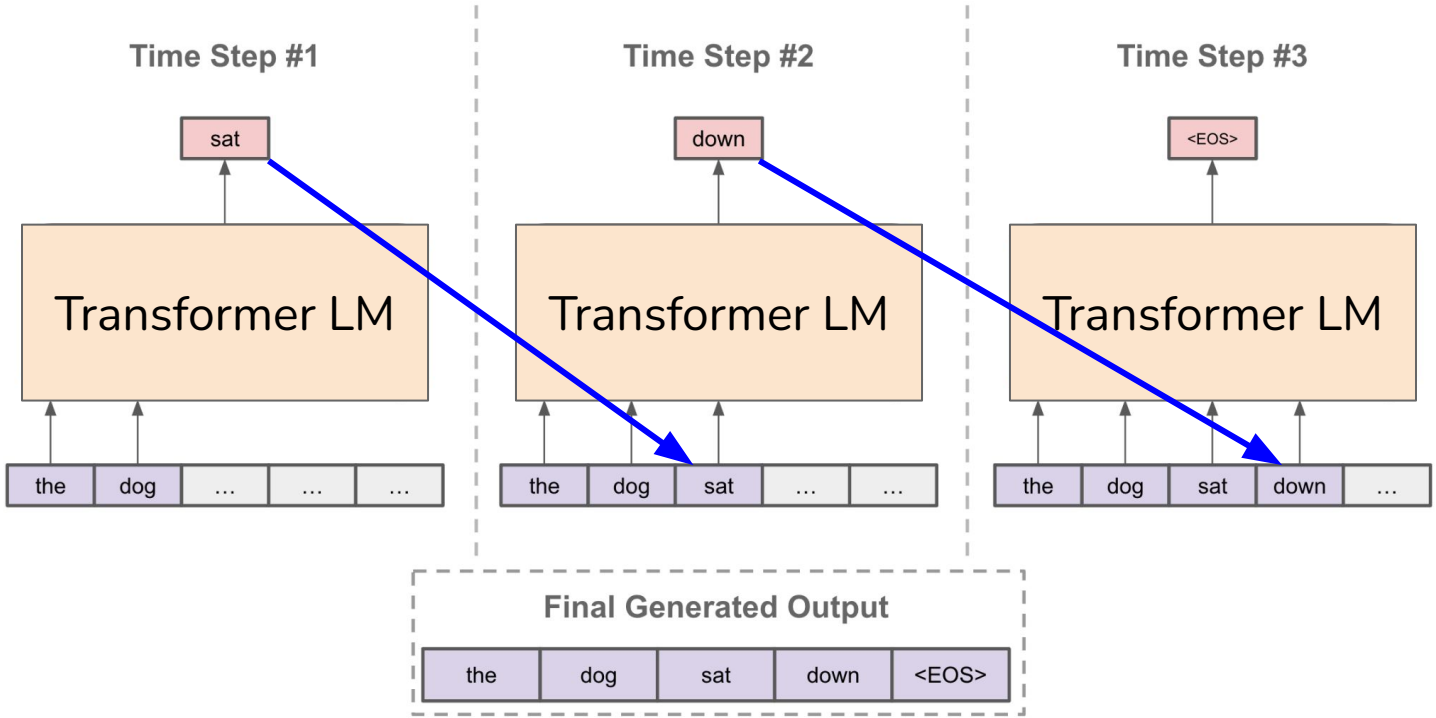
In inference, the model generates one token at a time. Given tokens $w_1, \ldots, w_i$:

1. Run the full forward pass to get $\hat{\boldsymbol{y}}_i = \text{softmax}(\boldsymbol{u}_i)$
2. Greedily select ($\text{argmax}_i \hat{\boldsymbol{y}}_i$) or sample next token w_{i+1} from this distribution.
3. Append w_{i+1} to the sequence and repeat.



Autoregressive generation: each predicted token becomes input for next step.

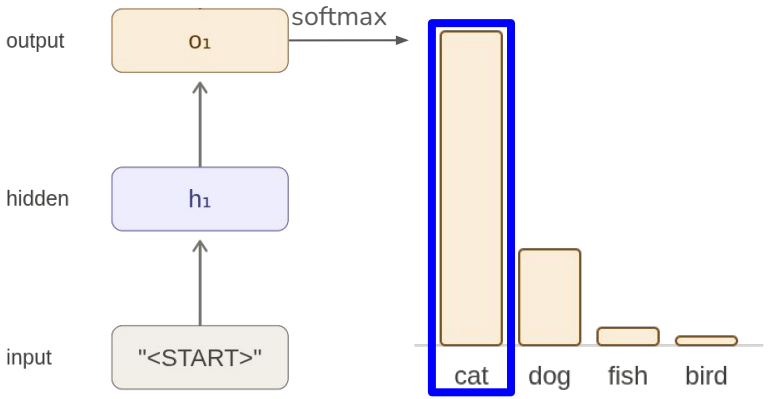
Autoregressive generation



Decoding

In text generation, **decoding is the process of selecting the next word or token based on the probability distribution produced by a language model.**

At every step:



<u>Greedy decoding:</u>

- At each step *t*, pick most probable token:

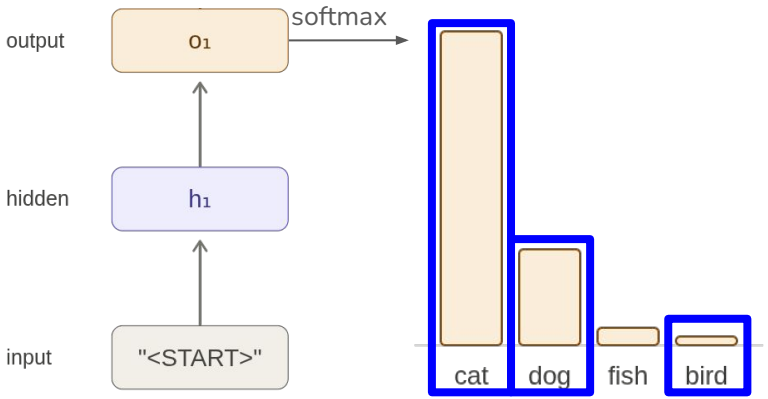
$$\text{argmax } P(w \mid \boldsymbol{x}_1, \ldots, \boldsymbol{x}_n, \boldsymbol{y}_1, \ldots, \boldsymbol{y}_{t-1})$$

- <u>Deterministic:</u> same seed ⇒ same output
- Simple, but tends to produce repetitive text and loops such as *“the the the…”*

Decoding

In text generation, **decoding is the process of selecting the next word or token based on the probability distribution produced by a language model.**

At every step:



Sampling

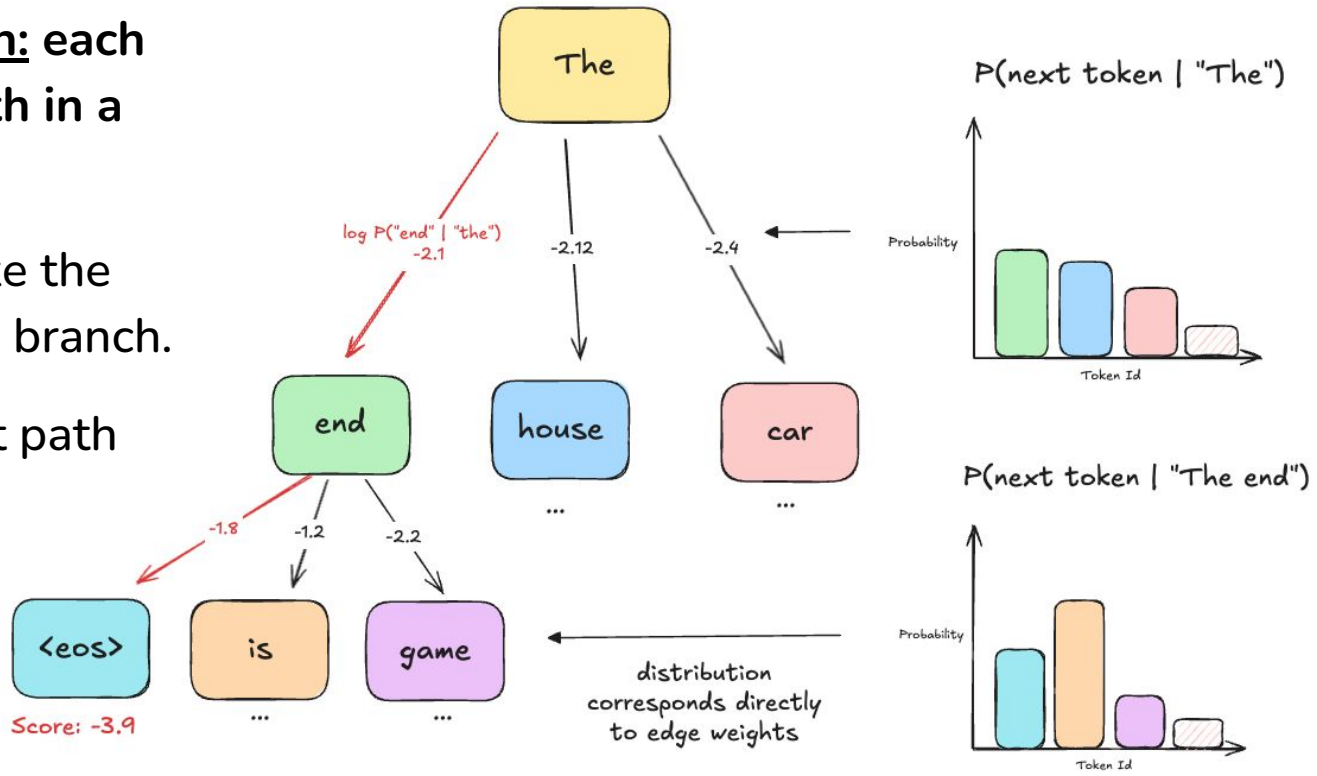
- *Draw* token from probability distribution instead of always taking the top one.
- Stochastic: many runs ⇒ different outputs
- More diverse text, but low-probability token can derail the sentence.

Decoding

<u>Decoding as search:</u> each generation is a path in a search tree.

Greedy: always take the highest-probability branch.

Sampling: different path every run.

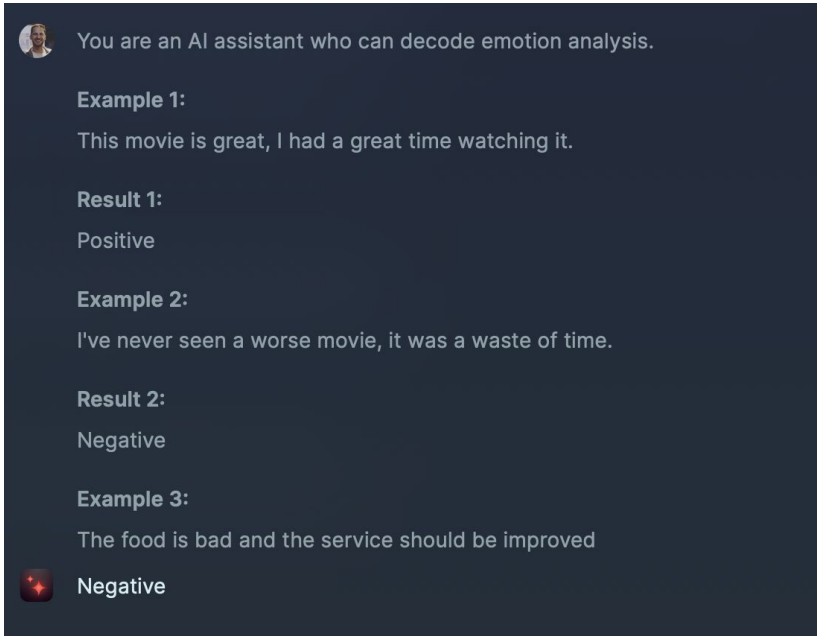


Test-Time Scaling

Test-Time Scaling

First way to scale inference: **scale the context (input)** provided to the model.

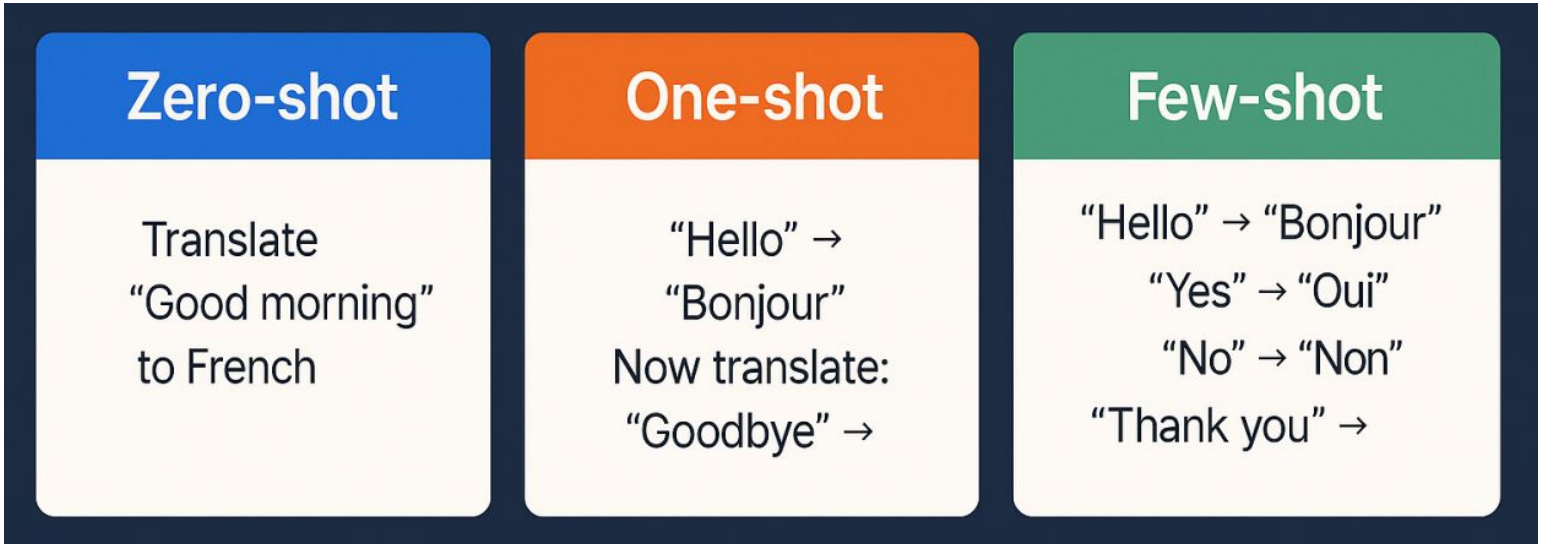
- **Goal:** condition generation y on more information in input/context x to improve task performance.
- **Few-shot prompting:** prompt with input-output examples, so model learns expected behaviour for task without any parameter updates.



In-context Learning

In-context learning: LLM learns and performs a new task based on examples provided in a prompt, without changing any of its internal parameters.

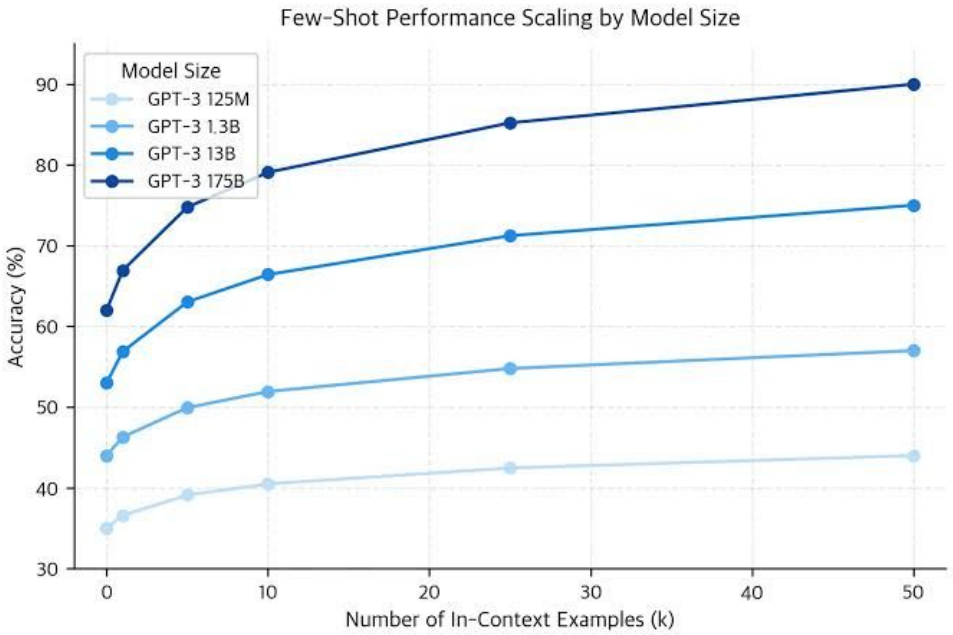
How many examples are necessary? Depends on the task...



In-context Learning

In-context learning emerges if pretraining is sufficiently scaled.

- Few-shot prompting can improve performance over zero-shot prompting.
- For many tasks, **increasing the number of examples improves performance** (up to a point).

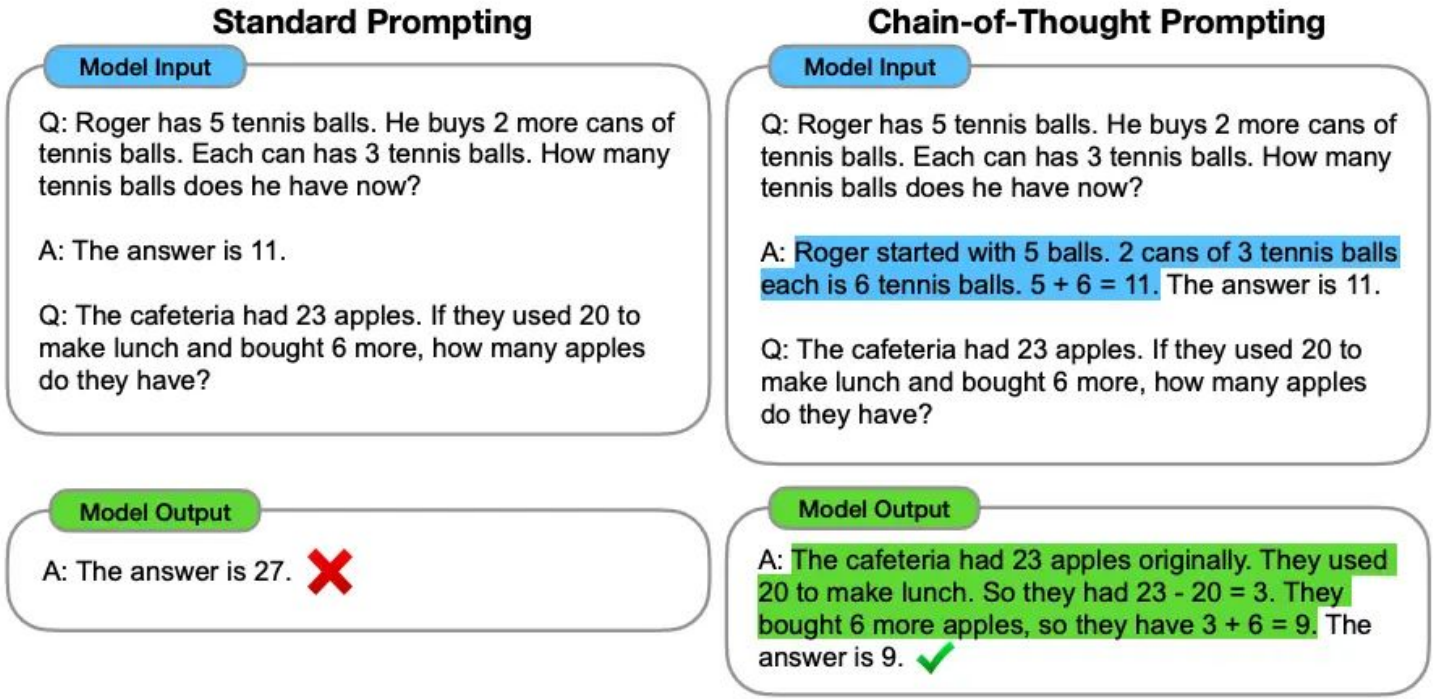


Chain-of-thought Prompting

How can we scale the context further?

CoT prompting: few-shot prompting where in-context examples provide intermediate steps in problem solving – answer is not generated directly, but after an explanation of reasoning steps required to arrive at answer.

Chain-of-thought Prompting



Chain-of-thought Prompting

Goal: encourage model to reason – **produce intermediate reasoning steps**.

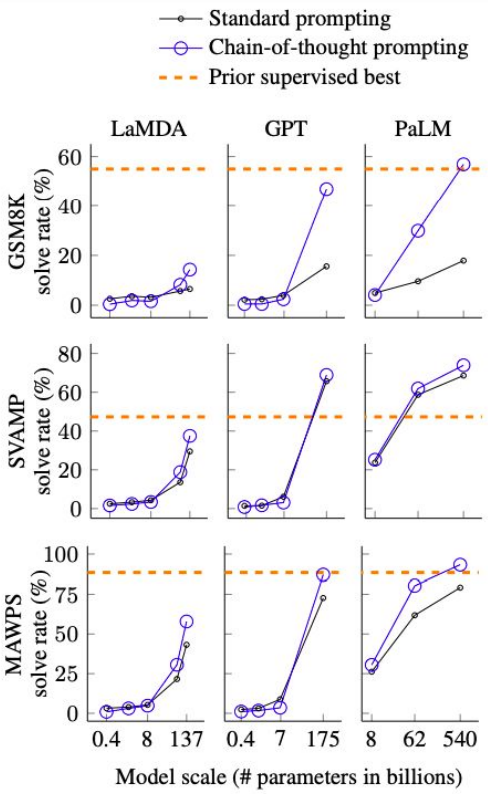
Why?

- Model can **break down complex tasks** into simpler subtasks, which can improve performance for hard tasks.
- Model can **learn the reasoning** steps required to solve a task.
- **Still no parameters being updated!** Learning without “training”.

Chain-of-thought Prompting

Findings:

- CoT prompting **emerges with scale** (only works for >100B parameter models).
- Biggest benefit over standard prompting is for **complex tasks** e.g. GSMK8K (maths problems).
- **Can even outperform task-specific finetuning** (training an LLM for a specific task), but doesn't require any parameter updates!



Chain-of-thought Prompting

Limitations:

- The model generates one chain-of-thought output – if it generates a wrong reasoning step, this derails the whole reasoning sequence!

  How to address this? Sample multiple chains-of-thought!

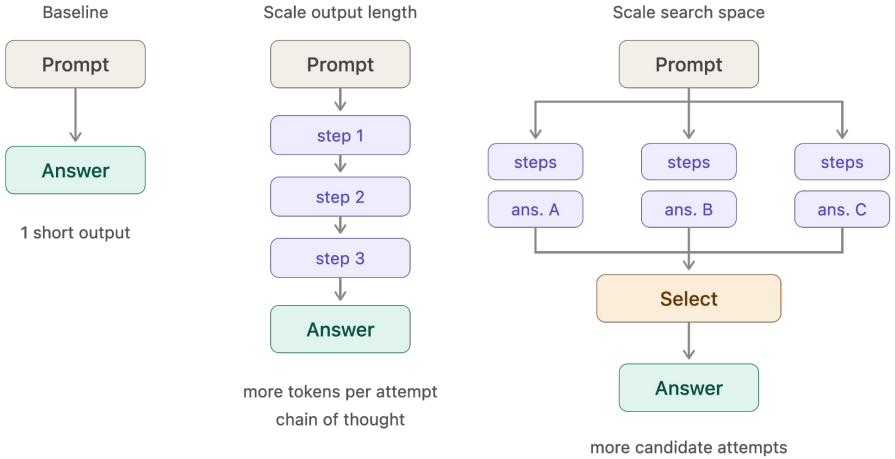
- We don't know if the reasoning trace is right – we have no way of evaluating the reasoning quality.

  How to address this? Introduce verifiers that assess answers/reasoning!

Test-Time Scaling

Few-shot prompting scales LLM input, but how can we scale generation?

- **Scale output length:** increase number of tokens generated during inference.
- **Scale search space:** expanding the set of candidate output sequences considered during search, so that higher-quality outputs can be found.

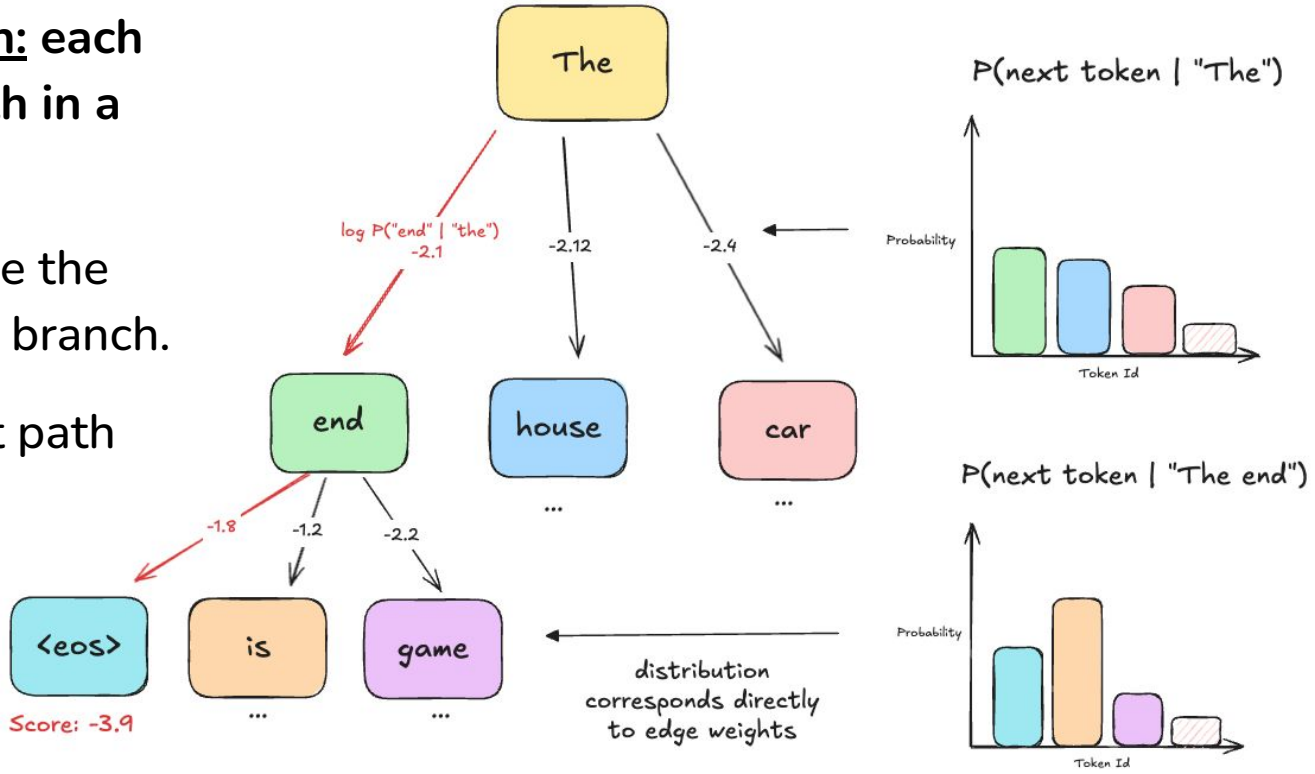


Decoding

<u>Decoding as search:</u> each generation is a path in a search tree.

Greedy: always take the highest-probability branch.

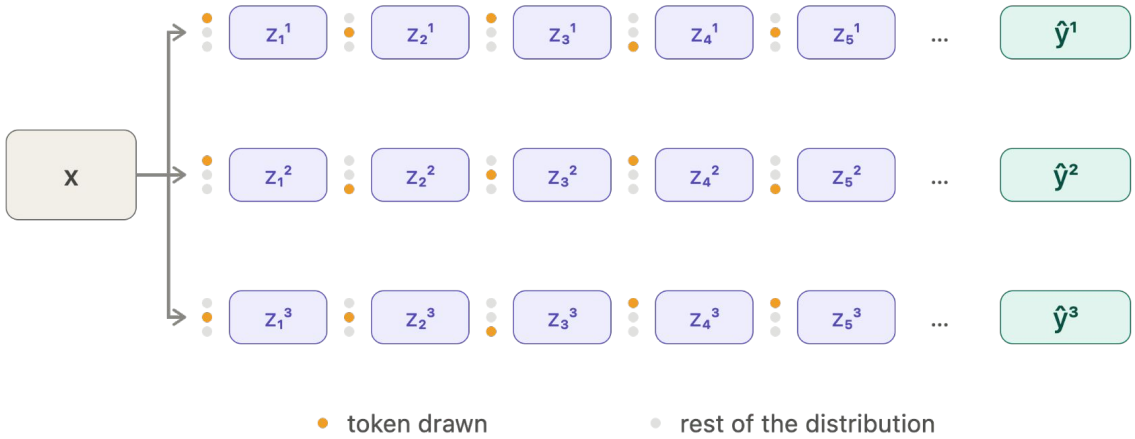
Sampling: different path every run.



Parallel scaling

Parallel scaling: for a given prompt x, sample multiple reasoning paths z.

- Each token in each path is generated with sampling, so it produces multiple different candidate generations (some of which end with the same answer).

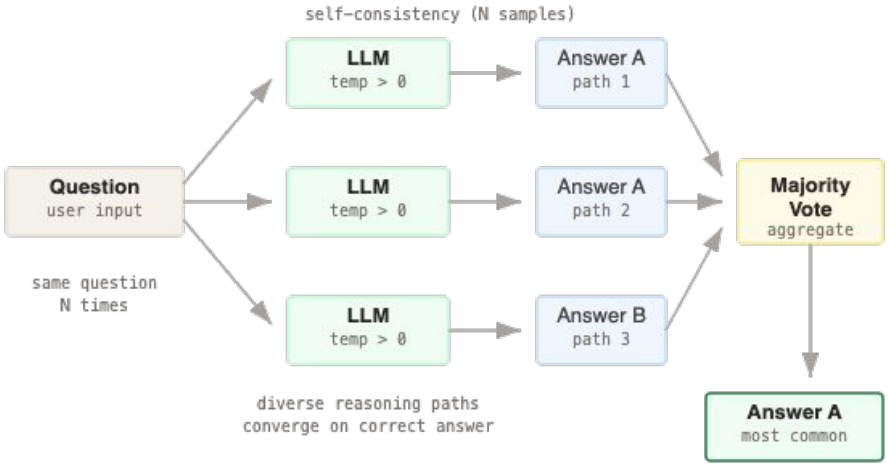


How do we determine the one final answer?

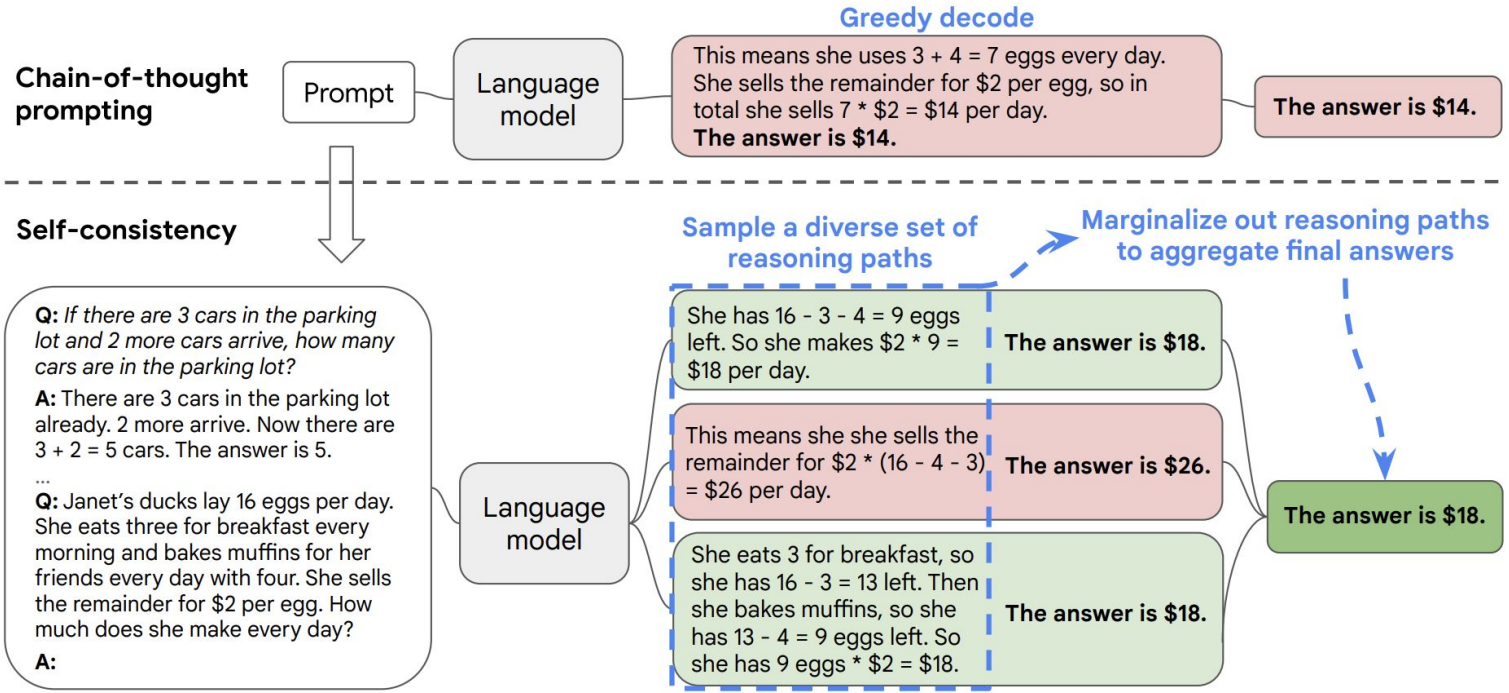
Self-consistency

Self-consistency: sample multiple reasoning paths (called **chain-of-thought rollouts**) and find the most common answer (**majority voting**)

Motivation: if multiple reasoning paths lead to the same answer, it is likely to be correct (if many approaches converge on one answer, this gives us confidence).



Self-consistency



Self-consistency

Advantages:

✅ Scales search space, which increases chance of finding good solution.

✅ Avoids relying on one CoT, which could be derailed by an invalid step.

✅ Incorporates reasoning diversity without requiring any further training!

Self-consistency

Disadvantages:

❌ Can only use for tasks with short answer e.g. maths, MCQs (not trivial to do majority voting for long, free-form answers).

❌ The model can arrive on **wrong answer through multiple reasoning paths!**

❌ The model can arrive on **right answer with invalid reasoning paths!**

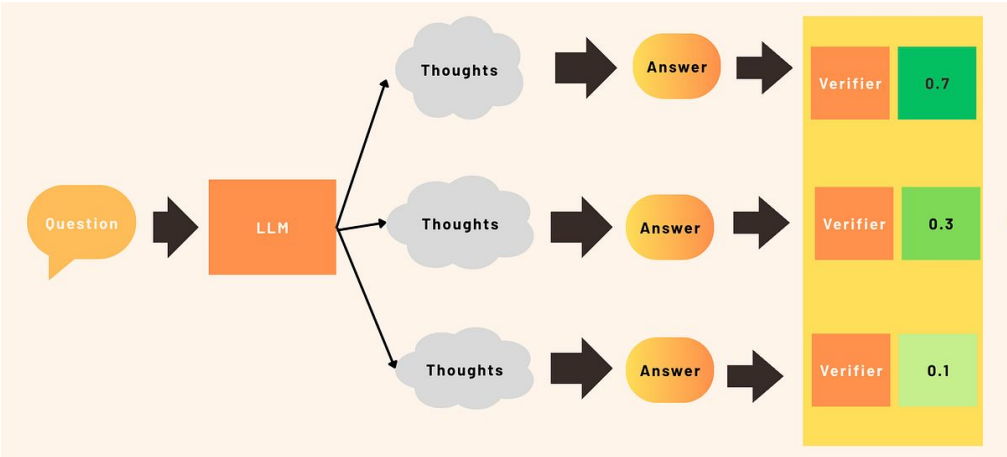
We need a way to evaluate the quality of answers or reasoning steps...

Verifiers!

Verification

Verifier: Model or function responsible for evaluating the quality, correctness, or utility of each candidate solution generated by the search algorithm.

- Scores each candidate answer, allows us to rank and select best answer.

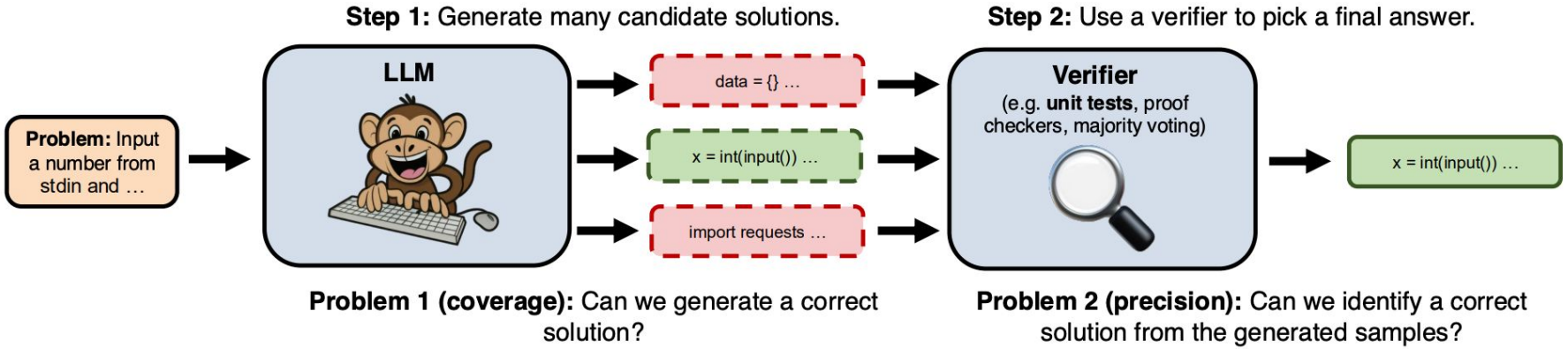


- Simpler to verify final answer, instead of entire reasoning path. But how??

Verification

Off-the-shelf tools as verifiers:

- For math, we can use proof checkers.
- For code, we can use compilers and unit tests.



Brown et al., "Large Language Monkeys: Scaling Inference Compute with Repeated Sampling." arXiv:2407.21787, 2024

Verification

Off-the-shelf tools as verifiers:

- For math, we can use proof checkers.
- For code, we can use compilers and unit tests.

Verifier models:

- **Train reward model** on question-answer data (answers scored by humans) to assign a score to each candidate answer.
- Use another **LLM as verifier** by prompting it to assess answer quality.

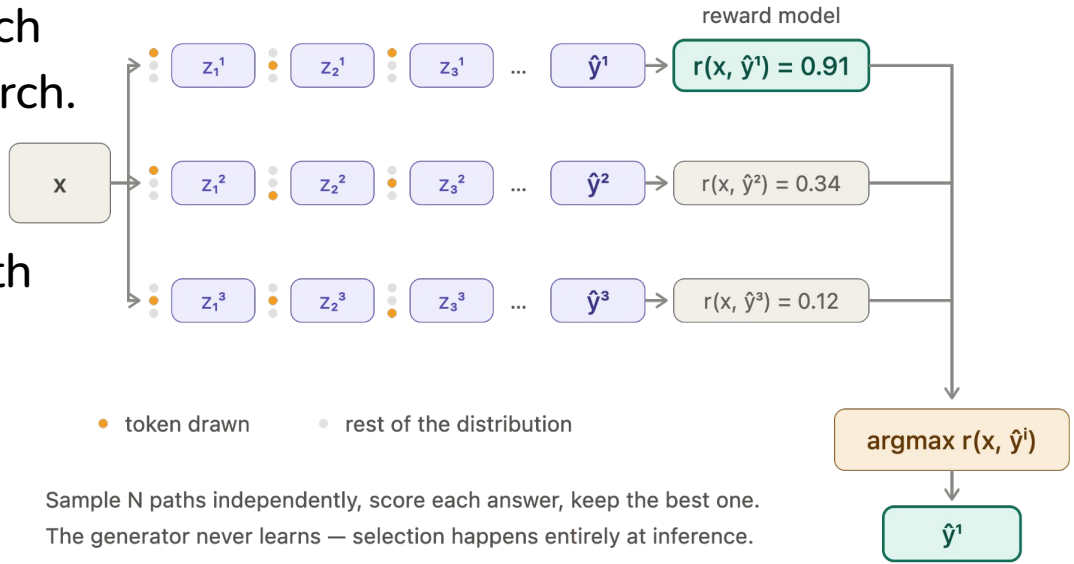
Best-of-N

(1) For input x, LLM generates N different output sequences $\{\hat{y}^1, ..., \hat{y}^N\}$.

*Outputs generated with any search algorithm e.g. sampling, beam search.

(2) Verifier scores each answer with its reward model r and selects the best one as the final answer:

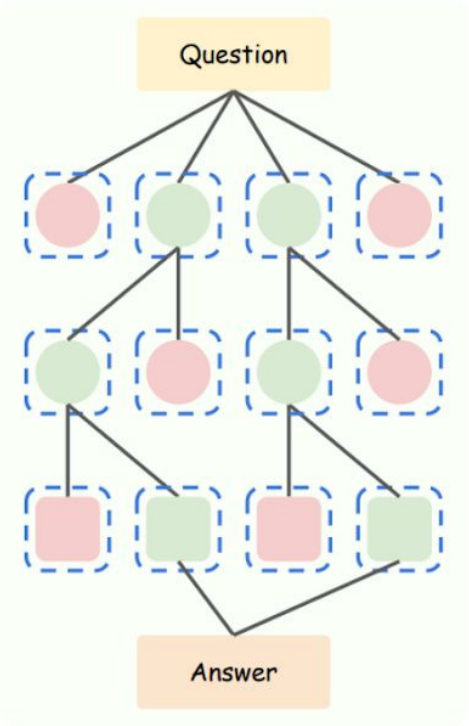
$$\hat{y}^1 = \max \{r(x, \hat{y}^1), ..., r(x, \hat{y}^N)\}$$



Verification

We can also do **step-level verifiers** e.g. **process reward models (PRM)** score every step of a reasoning chain, rather than just the final answer.

Enables step-level search: verifier is used to guide step-based search algorithm.



Test-Time Scaling

Two components for test-time search:

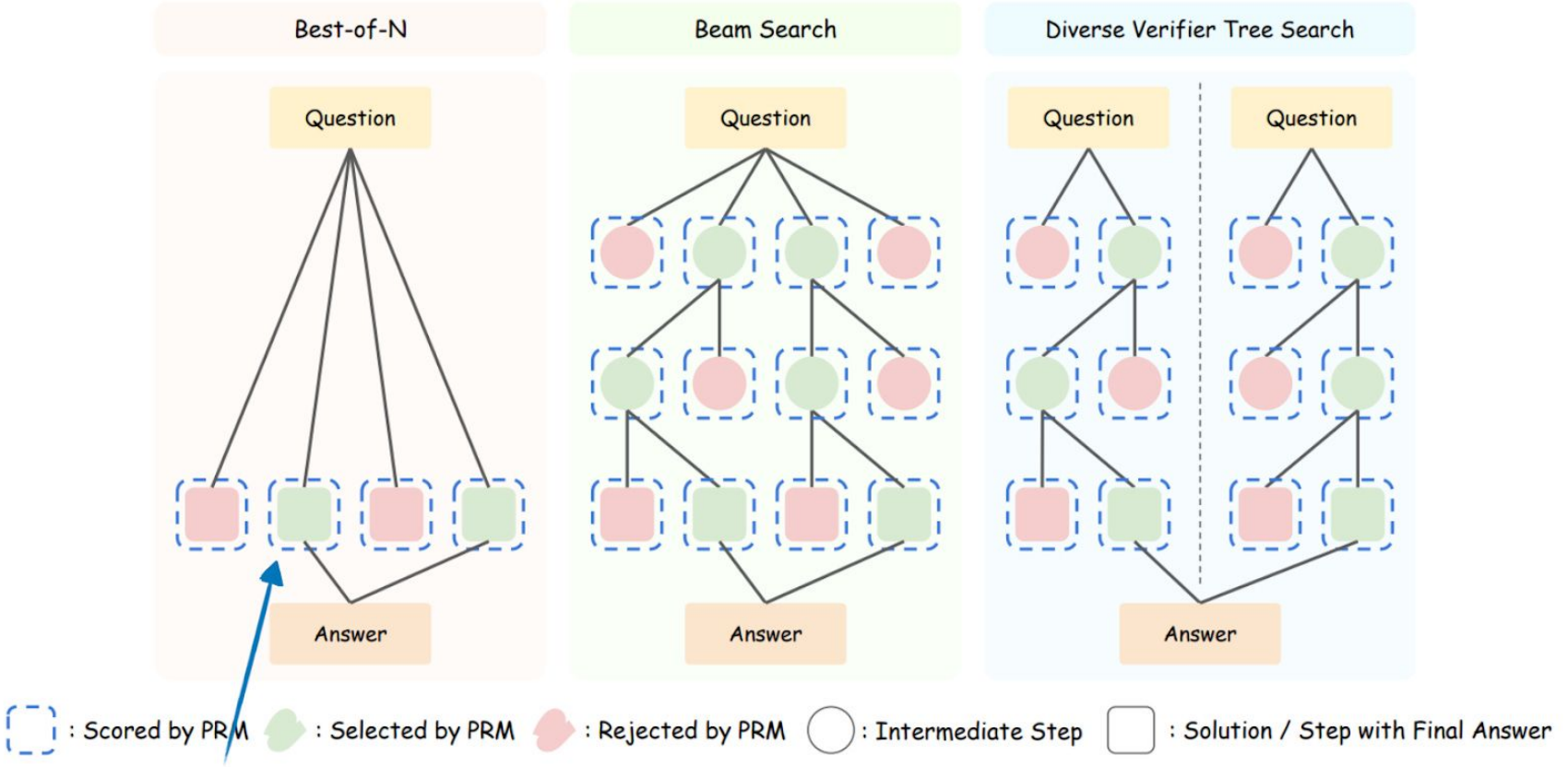
- **Search algorithm:** strategy used to explore the space of possible output sequences and generate a set of candidates.

  Can be simple sampling or more sophisticated search techniques.

- **Verifier:** Model or function that scores candidate solutions generated by the search algorithm.

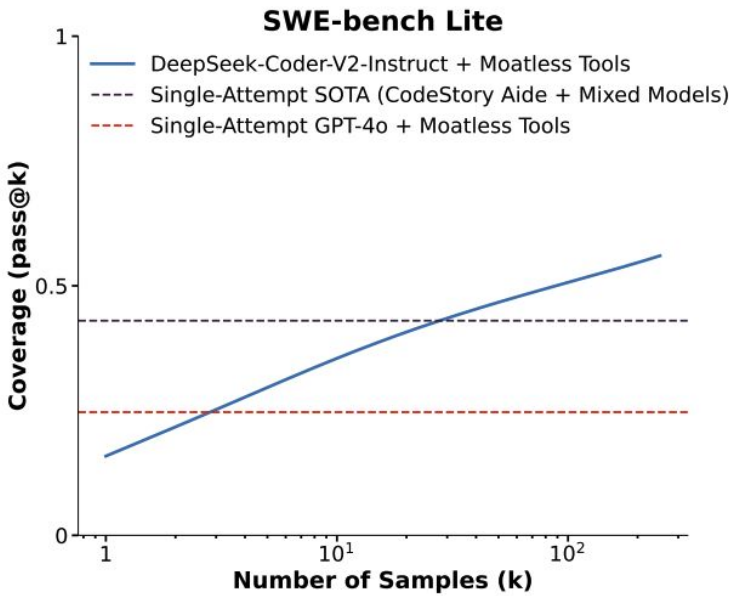
  As we've seen, there are many options for verifiers!

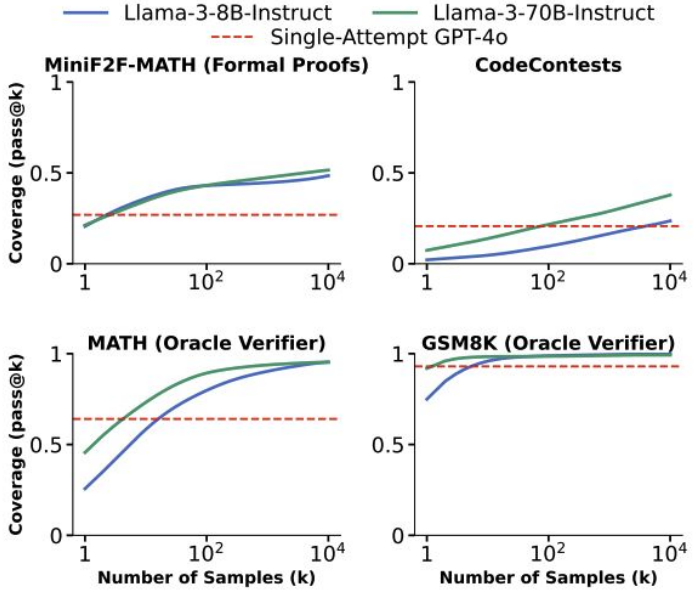
We can combine different search algorithms & verifiers:



Test-Time Scaling

Scaling test-time search improves performance on reasoning tasks.





Brown et al., "Large Language Monkeys: Scaling Inference Compute with Repeated Sampling." arXiv:2407.21787, 2024

Test-Time Scaling

We have to choose the search algorithm & verifier – what is the alternative?

Let the model learn how to reason!

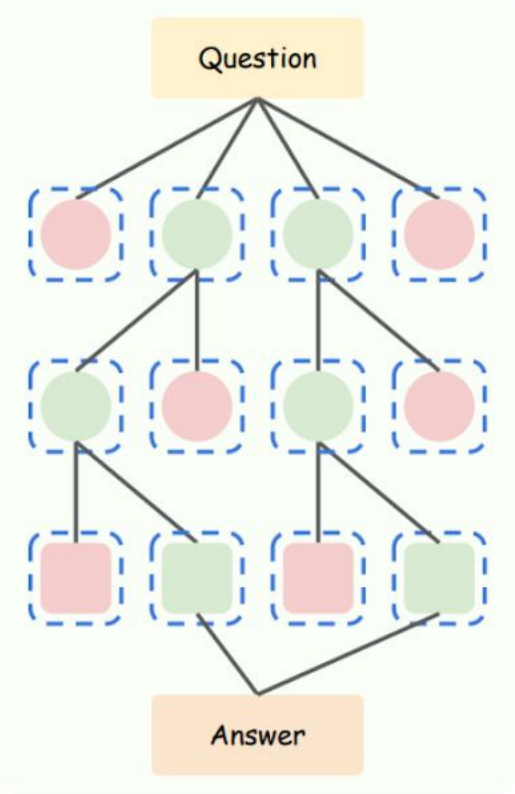
Training-based options:

- **Supervised finetuning (SFT)** on reasoning traces produced by humans.
- **Rejection sampling:** generate outputs, keep what verifier accepts, finetune on those, repeat. LLM generates own training data **(learns from successes)**.
- **RL:** generate outputs, score every attempt with verifier and update continuously **(learns from failures too)**.

Test-Time Scaling

Motivation:

- Train model to find good reasoning paths itself (to *learn* what the search + verifier did for us).
- Ideally, reduce reliance on multiple samples (one sample will be good enough).



RLVR

RLVR: Reinforcement Learning with Verifiable Rewards

- **Model generates its own attempts, each is scored by a verifier**, and weights are updated to make high-scoring attempts more likely and low-scoring attempts less likely.
- **Verifiable rewards = reward is a program** (e.g. unit tests, answer checker), not a model.
- Only works if correctness is checkable, which is why LLMs are so good at maths and coding!

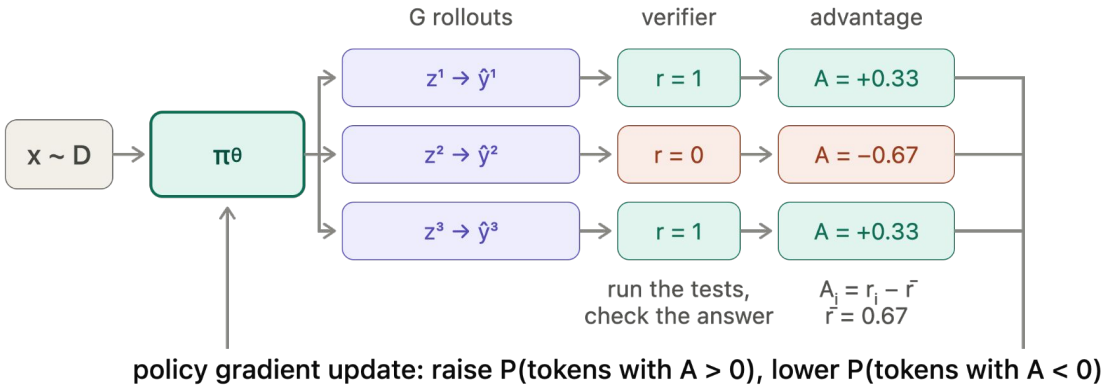
RLVR

state = prompt + tokens so far

action = next token

episode = one complete generation

reward = sparse, only at the end



- One reward for the whole attempt (no per-step labels).
- Every token shares credit equally, including some useless tokens. But across millions of updates, **useful reasoning steps will be rewarded.**
- **Model learns to reason by learning which steps lead to correct answers!**

DeepSeek R1-Zero

Start from pretrained model with no instruction tuning or reasoning examples.

For each problem, sample several CoT rollouts and apply RLVR based on:

- **Accuracy reward:** extract the boxed answer and match it against the correct answer or, for code, compile and run against test cases.
- **Format reward:** the reasoning must appear inside designated tags.

No human wrote a single reasoning trace.

The reward is one bit (0 = wrong, 1 = right) from a program. 🤯

DeepSeek R1-Zero

What emerged?

- Response length grew across training, reaching to thousands of tokens.
- **Sophisticated reasoning strategies like backtracking, self-verification, and re-deriving a result a second way – all without any demonstrations, just because the they correlate with being correct.** 🤯

This was the first time the o1 recipe was described openly and reproducibly (rare for this type of work).

DeepSeek R1-Zero

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to

Response: <think>

To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both $\cdots$

$\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2.$

Rearrange to isolate the inner square root term:

$(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$

$\ldots$

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$

We started with the equation:

$\sqrt{a - \sqrt{a + x}} = x$

First, let's square both sides:

$a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$

Next, I could square both sides again, treating the equation: $\cdots$

$\ldots$

DeepSeek-AI, DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. Nature, 2025.

Test-Time Scaling



RLVR is how frontier LLM agents are trained.

- **Verifier still used**, but now for RL training reward and not as an inference-time filter.
- **Reasoning is in the weights**, so one sample can produce a good reasoning trace.
- Self-consistency and best-of-N can still improve performance, but is not as necessary.

Reading

- **These slides are the main resource to study for the exam.**
- **Prescribed reading:** Section 5.3 in Foundations of Large Language Models by Xiao & Zhu: https://arxiv.org/pdf/2501.09223.

Optional reading if you want to learn about this in more depth:

- https://lilianweng.github.io/posts/2025-05-01-thinking/
- Chapter 6 in Foundations of Large Language Models by Xiao & Zhu: https://arxiv.org/pdf/2501.09223